TreeSage: Visualizing LLM Beam Search as an Interactive Typing Tutor

Team Members: Andrey Fedorov
Emails: afedorov@stanford.edu

1 Why

Most people fundamentally misunderstand how large language models work. When told that LLMs "just predict the next token" or "sample from a probability distribution," they often conclude that the models are essentially fancy random number generators. This project is a direct response to that misconception.

By removing both randomness and model choice from the generation process, TreeSage forces the user to navigate the model's actual probability space in real time. What remains — after stripping away sampling and autoregressive choice — is the underlying structure and coherence of the model's intelligence. The goal is to give users an intuitive, visceral understanding of what different LLMs are actually doing when they generate text.

2 What

TreeSage is an interactive typing tutor and visualization tool that displays the LLM's beam search as a live, scrolling horizontal tree.

As the user types:

- A tree of possible next tokens grows to the right from the top-left root.
- At each depth, the top-K most likely tokens are shown, ordered vertically by probability.
- Visible curved edges connect tokens to their continuations.
- The entire tree smoothly scrolls left as tokens are committed.
- In **Tutor Mode**, the user can only type or select tokens that exist in the current beam (with an error sound if they deviate).
- In **Free Mode**, the user can type freely while the tree visualizes predictions in the background.

The interface makes the model's latent decision process visible and tangible.

3 How

We will implement TreeSage as a web application using Svelte 5. The architecture uses:

- A hybrid rendering approach: HTML `divs` for text nodes and a single SVG layer for curved edges.
- Real-time beam search performed via LLM API calls (with caching where possible).
- A policy network (implemented in PyTorch) that learns when to stop expanding the tree and when to continue.
- Dynamic layout that fills the available screen space without vertical scrolling.

Early implementation will focus on the core typing + tree visualization experience. Later stages will add the reinforcement learning policy for adaptive questioning.